

ML Specification Report — playground-series-s4e1

Bank Customer Churn Prediction · our from-scratch agent (GBDT pool + tree-search v3)

Figures grounded in the competition's `facts.json`, `eda_summary.json`, `STATUS.md`, and `experiments_tree_v3.json`.

Overview

The task is to rank 110,023 bank customers by churn probability (metric: ROC-AUC). We solve it with a from-scratch pipeline — a pool of gradient-boosted trees (LightGBM / XGBoost / CatBoost) refined by a four-tier ablation and a 60-node tree search that ends on a 37-member blend. Our champion scores **CV AUC 0.894393**; the NVIDIA reproduce-agent, copying a public CatBoost kernel (leak-free), reaches 0.8984 — NVIDIA ahead by ~0.4%.

Why it matters. Retaining an existing customer is far cheaper than acquiring a new one, so ranking churn risk lets a bank direct retention offers to the customers where they pay off most.

Data

Purpose of Data. Predict whether a bank customer churns (leaves the bank) — a binary-classification Playground Series episode. **Data Format** is clean **tabular CSV**, a mix of numeric and categorical columns. **Data Volume** is **165,034 training rows / 110,023 test rows**, with 12 raw predictive features once `id` and the target are removed.

Data Quality is high: **no missing values** in either split, **0 duplicate rows**, and no train ↔ test mean shift. The one modelling wrinkle is class imbalance — **21.2% of customers churned** (`Exited=1`), which shapes the choice of metric (AUC) and the `is_unbalance` ablation in training. The strongest EDA signals, all consistent with domain intuition, were:

- **Age** is the dominant predictor ($r \approx +0.34$); churn peaks in the 40s–50s.
- **NumOfProducts** is highly non-linear: 3–4 products → ~88% churn, 1 product → 35%, 2 products → 6%.
- **Geography**: Germany 37.9% churn vs France 16.5% / Spain 17.2%; **Gender**: female 28.0% vs male 15.9%.
- **Surname** carries geographic/cultural signal (e.g. "McGregor" 75.9% churn vs "Davey" 0%), usable via target encoding.

Annotation Guidelines. The label is `Exited` $\in \{0, 1\}$ (1 = churned). Submissions are **churn probabilities** scored by AUC — a ranking, not a hard 0/1 decision.

Feature Set. From the 12 raw columns we engineer **28 features**, grouped below:

Group	Features
Dummies	is_female , geo_Germany , geo_Spain
Age	age_sq , age_decade , age_40_60 (peak-churn band)
Balance	has_balance , balance_salary_ratio
Products	products_gt2 , products_eq1
Interactions	age_x_products , age_x_balance , age_x_active , geo_gender , active_x_balance , active_x_products , credit_age_ratio
Encoded	surname OOF target-encoding (smoothing = 20)

Splitting strategy. A single canonical **Stratified 5-Fold CV (seed = 42)**, shared across all four tiers so scores are directly comparable. The critical safeguard is **fold-safe target encoding** — the surname encoding is computed on the same fold partition used for training (TE-fold == model-fold), closing a subtle leakage path that an earlier February run had left open. The leaderboard is closed (late submission returns 403), so all scores here are **CV-only**; the prior-season submission (CV 0.897 → Private 0.892, gap ≈ 0.005) anchors CV trustworthiness.

Models & Architecture

Purpose of Architecture. A binary churn classifier that maximizes ROC-AUC. **Architecture Type** is a **gradient-boosted decision-tree ensemble** — LightGBM as the primary learner, with XGBoost and CatBoost members and a linear/logistic member, combined by a weighted blend; the champion is a **37-member mega-blend** discovered at tree-search node #44.

Each member takes the same **Input Format** — a numeric feature matrix with categoricals encoded (dummy / label / OOF target-encoding) — of **Input Dimension 28 features** per row (a 1-D vector; there is no spatial/sequence structure).

Architecture Description. Each GBDT member is an additive ensemble of decision trees grown by histogram-based, leaf-wise gradient boosting (representative LGB: num_leaves = 63, ~159 boosting iterations). The blend layer sits on top as a convex weighted average of the members' out-of-fold predictions, with weights found by Dirichlet sampling / coordinate-ascent / NNLS search. **Model Complexity** is therefore best expressed as trees $\times$ leaves rather than a dense parameter count: a single LGB member $\approx 159 \times 63 \approx 10k$ leaves, and the champion blends up to 37 such members.

Training procedures

Training proceeds as a **four-tier ablation** on the shared 5-fold split, each tier adding capability and measured on the same OOF scale:

Tier	Configuration	OOF AUC
1	generic 3-model blend (raw label-encoded columns)	0.8922
2	skill pipeline: 28 features + fold-safe surname TE, 3-model blend	0.894302

Tier	Configuration	OOF AUC
3	+ linear iteration (is_unbalance ablation / Optuna 50-trial tune / seed-bag), 5-way blend	0.894354
4	+ tree-search harness_v3 (60 nodes, node #44 = 37-member mega-blend)	0.894393

The **Loss Function** is binary log-loss (LightGBM `binary` objective) with model selection on AUC; the `is_unbalance` class-weighting was ablated and *removing* it was marginally better ($\Delta -0.000415$). The **Optimization Algorithm** is gradient boosting (histogram, leaf-wise) — **not** SGD/ADAM — with blend weights optimized by coordinate-ascent / NNLS. The **Learning Rate** is the boosting shrinkage (LGB `learning_rate` = 0.05); there is **no Learning Rate Scheduler** (N/A — *GBDT convergence is governed by CV early-stopping, not a schedule*) and **no Batch Size** (N/A — *full-dataset histogram boosting, not mini-batched*).

Training Duration for the tier-4 search was ~2,687 s of wall-clock over 60 nodes; the Optuna tune was 50 trials / 678 s and a solo LGB fits in ~159 iterations. **Training Memory** was not explicitly capped — a $165k \times 28$ float matrix is trivial on the single arm64 CPU machine, peak not separately measured. **Transfer Learning** is N/A (*no pretrained weights*); its analogue here is **cross-season experience injection** — a distilled S3 recipe ("Optuna fold-proxy tune → pool → seed-bag") carried into S4. **Data Augmentation** is N/A (*tabular*); the analogues are feature engineering (the 28 features) and seed-bagging.

Reproducibility Standards are strict: fixed seeds (fold seed = 42; Optuna and seed-bag seeds logged), LightGBM determinism flags (`deterministic`, `force_row_wise`, fixed `num_threads`), and an H-1 resume contract that reloads the search state with bit-level seed-score matching. A faithful replay reproduces the committed champion to $\leq 1e-4$.

Inference procedures

Decision Threshold is N/A — AUC is a ranking metric, so we submit raw probabilities and never threshold (a 0.5 cut would only be needed if hard labels were required). **Inference Duration** was not separately profiled; GBDT scoring of the 110,023 test rows is sub-second per member on CPU, and **Inference Memory** is negligible (tree inference is lightweight, not capped).

Evaluation & Benchmarking

Task for Performance Evaluation. Rank the test customers by churn probability. **Performance Metrics.** ROC-AUC is the sole competition metric; our tier-4 champion reaches **CV OOF AUC 0.894393**, climbing $0.8922 \rightarrow 0.894302 \rightarrow 0.894354 \rightarrow 0.894393$ across the four tiers. The February prior-season baseline (same data family, before the TE-leak fix) posted Public LB 0.88716 / Private 0.89179 — reference only, as this run is CV-only.

Performance Benchmarking. The natural comparator is the **NVIDIA reproduce-agent**, which copies the strongest public kernel:

Agent	Approach	CV AUC	Note
NVIDIA	reproduces <code>aspillai/bank-churn-catboost</code> (leak-free)	0.8984	winner

Agent	Approach	CV AUC	Note
Our agent	from-scratch GBDT pool + tree-search	0.894393	−0.4%

NVIDIA leads by ~0.4%. The honest caveat: the public kernel's headline score exploited a **test-set leak** that we deliberately excluded for a fair, generalizable comparison — so both numbers here are leak-free. On the newer seasons (S5–S6) this reproduce-vs-originate gap collapses to a dead heat, which is analyzed separately in the four-axis evaluation; for this S3-era-style churn problem the well-optimized public kernel retains a small edge.